

A Project Report On

Student Pass/Fail Prediction Using Machine Learning

Submitted in partial fulfillment of the requirement for the
award of the degree

MASTER OF COMPUTER APPLICATIONS (MCA)

From

Marwadi University

Academic Year 2026 – 27

VIDHI NIMAVAT (92500584005)
NAZNIN SHERASIYA (92500584096)

Internal Guide

DR. DIVYAKANT MEVA



Marwadi
University
Marwadi Chandarana Group





Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FoCA)

Certificate

**This is to certify that the project work entitled
Student Pass/Fail Prediction Using Machine
Learning**

**submitted in partial fulfillment of the requirement for
the award of the degree of
Master of Computer Applications (MCA)
of the**

Marwadi University

is a result of the bonafide work carried out by

**VIDHI NIMAVAT (92500584005)
NAZNIN SHERASIYA (92500584096)**

during the academic year 2026-27

Faculty Guide

HOD

Dean

DECLARATION

We hereby declare that this project work entitled **Student Pass/Fail Prediction Using Machine Learning** is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place:

Date:

VIDHI NIMAVAT (92500584005) Signature:_____

NAZNIN SHERASIYA (92500584096) Signature:_____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **DR. DIVYAKANT MEVA**, the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his guidance, suggestions and expertise at every stage. A part from that his valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rijwan Khan** sir and HoD **Dr. Sunil Bajaja** sir for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

VIDHI NIMAVAT (92500584005) Signature:_____

NAZNIN SHERASIYA (92500584096) Signature:_____

CONTENTS

Chapters	Particulars	Page No.
1	Introduction	1
2	Dataset Description 2.1 Overview 2.2 Feature Matrix (All Columns) 2.3 Descriptive Statistics 2.4 Technical Indicators Purpose & Rationale	2
3	Dataset Preprocessing & Feature Engineering 3.1 Steps Performed 3.2 Train-Test Split 3.3 Feature Scaling	7
4	Machine Learning Models Used 4.1 Logistic Regression (Baseline) 4.2 Decision Tree Classifier 4.3 Random Forest Classifier (Ensemble — Bagging) 4.4 K-Nearest Neighbors (KNN) 4.5 Support Vector Machine (SVM)	10
5	Evaluation Metrics	15
6	Technology Stack	17
7	CODE	18
8	Visualization	28
9	Conclusion	39

FIGURE INDEX

Sr. No.	Figure No.	Particulars	Page No.
1	Figure 1	Pass/Fail Distribution	28
2	Figure 2	Pass/Fail Percentage	29
3	Figure 3	Distribution of Numerical Features	32
4	Figure 4	Numerical Features vs Pass/Fail — Boxplots	35
5	Figure 5	Accuracy Comparison of Classification Algorithms	36
6	Figure 6	Confusion Matrices of All Five Classification Algorithms	36
7	Figure 7	Final Confusion Matrix of Best Performing Model	37
8	Figure 8	Final Model Performance	37
9	Figure 9	Final Model Metric Comparison	38

TABLE INDEX

Sr. No.	Table No.	Particulars	Page No.
1	Table 1	Table 1: Dataset Overview	2
2	Table 2	Table 2: Feature Matrix — All Columns with Description, Type & Unit	4
3	Table 3	Table 3 :Descriptive Statistics (Count, Mean, Min, Max, Std Dev, Percentiles)	4
4	Table 4	Table 4: Train-Test Split (Rows & Date Period)	9
5	Table 5	Table 5: Feature Scaling Strategy Per Model	9
6	Table 6	Table 6: Evaluation Metrics (Formula & Interpretation)	16
7	Table 7	Table 7: Technology Stack (Component & Library)	17

1. Introduction to Project Definition

The **Student Performance Prediction** project is a Machine Learning-based system developed to predict whether a student is likely to pass or fail based on different academic and behavioral attributes. The project uses historical student data and applies classification techniques to discover patterns that may be associated with academic outcomes. By analyzing these patterns, the system can provide a prediction of the student's expected result.

The project considers various factors, including **study hours per week, attendance rate, previous grades, participation in extracurricular activities, parent education level, assignment completion rate, mock test score, and daily self-study hours**. These attributes are processed and prepared before being supplied to Machine Learning models. Data preprocessing is performed to handle missing or invalid values and to ensure that the dataset is suitable for model training.

Five classification algorithms are considered in the project: **Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)**. Initially, baseline models are developed to establish a performance reference. The models are then improved using hyperparameter tuning with cross-validation. Their performance is evaluated using measures such as **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**.

2. Dataset Description

2.1 Overview

Property	Details
Dataset Name	Student Performance Prediction Dataset
File	student_performance_prediction_50000_records_10_columns.csv
Total Records	50,000 records
Total Features	8 input features + 1 target variable
Input Features	Study Hours per Week, Attendance Rate, Previous Grades, Participation in Extracurricular Activities, Parent Education Level, Assignment Completion Rate, Mock Test Score, Daily Self-Study Hours
Target Variable	Passed — indicates whether the student passed or failed
Target Classes	Yes / No
Task Type	Supervised Classification
Algorithms Used	Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Support Vector Machine (SVM)

Table 1: Dataset Overview

The dataset contains a large collection of **student academic and behavioral records** used to predict students' academic outcomes. Each record represents a student's learning-related information, including study time, attendance, previous grades, assignment completion, mock test performance, self-study habits, and other educational factors.

2.2 Feature Matrix (All Columns)

#	Column Name	Description	Type	Unit
1	Student ID	Unique identifier assigned to each student record	Identifier	—
2	Study Hours per Week	Total amount of time a student spends studying during a typical week	Numerical	Hours
3	Attendance Rate	Percentage of classes attended by the student	Numerical	%
4	Previous Grades	Academic performance obtained by the student in previous examinations or assessments	Numerical	Score / %
5	Participation in Extracurricular Activities	Indicates whether the student participates in extracurricular activities	Categorical	Yes / No
6	Parent Education Level	Educational qualification or education category of the student parent(s)	Categorical	Category
7	Assignment Completion Rate	Percentage of assigned academic work successfully completed by the student	Numerical	%
8	Mock Test Score	Score obtained by the student in practice or mock examinations	Numerical	Score / %

9	Daily Self-Study Hours	Average number of hours spent by the student on self-study each day	Numerical	Hours / Day
10	Passed	Indicates the final academic outcome that the model is designed to predict	Categorical / Target	Yes / No

Table 2: Feature Matrix — All Columns with Description, Type & Unit

2.3 Descriptive Statistics

Statistic	Study Hours per Week	Attendance Rate	Previous Grades	Assignment Completion Rate	Mock Test Score	Daily Self-Study Hours
Count	48,005	48,008	48,006	50,000	50,000	50,000
Mean	10.60	74.99	65.44	70.15	65.25	1.52
Min	-12.30	-14.30	3.00	0.00	0.00	0.50
25th Percentile	6.50	61.70	55.10	59.00	54.00	0.90
Median	10.00	75.00	65.30	71.00	65.00	1.50
75th Percentile	13.60	88.70	75.10	83.00	77.00	2.00
Max	78.00	150.20	200.00	100.00	100.00	6.10
Std Dev	7.91	19.96	16.20	17.14	16.59	0.77

Table 3: Descriptive Statistics (Count, Mean, Min, Max, Std Dev, Percentiles)

2.4 Technical Indicators — Purpose & Rationale

- **Academic Average:**

Combines the student's Previous Grades, Assignment Completion Rate, and Mock Test Score to represent the student's overall academic performance. A higher academic average indicates stronger and more consistent academic performance.

- **Assessment Average:**

Calculates the average of Assignment Completion Rate and Mock Test Score. This feature provides a combined measure of how well a student performs in assessments and completes academic tasks.

- **Total Study Time Equivalent:**

Combines Study Hours per Week with Daily Self-Study Hours to represent the student's overall study effort. This helps the model understand the relationship between regular study time and the likelihood of passing.

- **Study Attendance Interaction:**

Represents the interaction between Study Hours per Week and Attendance Rate. It helps capture whether students who spend more time studying and attend classes regularly have a higher likelihood of passing.

- **Study Performance Interaction:**

Combines Study Hours per Week with Previous Grades to identify the relationship between current study effort and previous academic performance.

- **Attendance Performance Interaction:**

Combines Attendance Rate and Previous Grades to capture the relationship between class attendance and the student's previous academic performance.

- **Assignment Mock Interaction:**

Combines Assignment Completion Rate and Mock Test Score to represent the relationship between assignment completion and test performance. This can help the model identify students who consistently perform well in different types of assessments.

3. Dataset Preprocessing & Feature Engineering

3.1 Steps Performed

- **Data Loading:**

The Student Performance Prediction dataset was loaded using pandas. The dataset contains **50,000 student records and 10 original columns**, including study hours, attendance rate, previous grades, extracurricular participation, parent education level, assignment completion rate, mock test score, daily self-study hours, and the target variable *Passed*.

- **Data Inspection:**

The dataset was inspected for data types, missing values, duplicate records, and invalid numerical values. No duplicate records were found. Missing values were present in several columns, including Study Hours per Week, Attendance Rate, Previous Grades, Participation in Extracurricular Activities, Parent Education Level, and Passed.

- **Invalid Value Handling:**

Invalid numerical values were identified and replaced with NaN before applying the missing-value treatment. Invalid values were found in Study Hours per Week, Attendance Rate, and Previous Grades. A total of **898 invalid Study Hours values, 3,943 invalid Attendance Rate values, and 463 invalid Previous Grades values** were replaced.

- **Missing Value Treatment:**

Missing values were handled during preprocessing so that the dataset could be used for machine learning. After cleaning and removing records without a valid target value, **47,469 records** were available for modelling.

- **Target Variable Encoding:**

The target variable **Passed** contains two classes: *No* and *Yes*. These classes were encoded numerically as **0 and 1** for machine learning. The target distribution contained 23,880 *No* records and 23,589 *Yes* records.

- **Categorical Feature Encoding:**

Categorical variables such as Participation in Extracurricular Activities and Parent Education Level were converted into numerical representations so that they could be processed by the machine learning algorithms.

- **Feature Engineering:**

Seven additional features were created from the existing academic and behavioural variables:

- Academic Average
- Assessment Average
- Total Study Time Equivalent
- Study Attendance Interaction
- Study Performance Interaction
- Attendance Performance Interaction
- Assignment Mock Interaction

3.2 Train -Test Split

An **80/20 train-test split** was used to divide the cleaned dataset into training and testing sets. The split was performed after preprocessing and feature engineering, with the test set kept separate for evaluating the models.

Split	Rows	Percentage
Training Set	37,975	80%
Testing Set	9,494	20%
Total	47,469	100%

Table 4: Train-Test Split (Rows & Date Period)

3.3 Feature Scaling

Model	X Scaling	Y Scaling
Logistic Regression	StandardScaler	None
Decision Tree	None	None
Random Forest	None	None
K-Nearest Neighbors (KNN)	StandardScaler	None
Support Vector Machine (SVM)	StandardScaler	None

Table 5: Feature Scaling Strategy Per Model

4. Machine Learning Models Used

4.1 Logistic Regression (Baseline)

- **Algorithm:** Logistic Regression —
`sklearn.linear_model.LogisticRegression`
- **How it works:** Logistic Regression estimates the probability that a student belongs to one of the two target classes, **Passed = Yes** or **Passed = No**. It uses a logistic (sigmoid) function to convert the weighted combination of input features into a probability between 0 and 1.
- **Purpose:** Logistic Regression was used as the **baseline classification model**. Its performance provides a reference point for comparing the other machine learning algorithms.
- **Features:** The model uses the cleaned original features along with the engineered academic and study-related features.
- **Scaling:** Features were standardized using `StandardScaler` because Logistic Regression can be affected by differences in feature magnitude.
- **Hyperparameter Tuning:** The regularization parameter `C` and class-weight options were evaluated using cross-validation. The best configuration was selected based on validation performance.
- **Model-Specific Analysis:** Accuracy, Precision, Recall, F1 Score, confusion matrix, and classification report were used to evaluate the model.

4.2 Decision Tree Classifier

- **Algorithm:** Decision Tree —
sklearn.tree.DecisionTreeClassifier
- **How it works:** A Decision Tree creates a series of decision rules by splitting the dataset according to feature values. Each split attempts to separate students into the **Passed** and **Failed** classes as effectively as possible. The final prediction is obtained from the leaf node reached by a student.
- **Purpose:** The Decision Tree was included because it can model **non-linear relationships** between student characteristics and academic outcomes and is relatively easy to interpret.
- **Scaling:** No feature scaling is required because Decision Trees make decisions using feature-based thresholds.
- **Hyperparameter Tuning:** Parameters such as:
 - criterion
 - max_depth
 - min_samples_split
 - min_samples_leaf
 - class_weight

were evaluated using cross-validation.

- **Model-Specific Analysis:** A confusion matrix and classification metrics were used to examine how well the tree classified students into the two outcome classes.

4.3 Random Forest Classifier (Ensemble — Bagging)

- **Algorithm:** Random Forest — `sklearn.ensemble.RandomForestClassifier`
- **How it works:** Random Forest builds multiple Decision Trees using different subsets of the training data and features. Each tree produces a prediction, and the final classification is determined by combining the predictions of the individual trees.
- **Purpose:** Random Forest was selected as an **ensemble learning method** to improve predictive stability and reduce the overfitting that can occur with a single Decision Tree.
- **Scaling:** No feature scaling is required because Random Forest is a tree-based algorithm.
- **Hyperparameter Tuning:** The following parameters were evaluated:
 - `n_estimators`
 - `max_depth`
 - `min_samples_split`
 - `min_samples_leaf`
 - `max_features`
 - `class_weight`
- **Best Configuration:** During cross-validation, the best Random Forest configuration used **150 trees**, `min_samples_leaf = 5`, `max_features = 'log2'`, and `class_weight = 'balanced'`.
- **Model-Specific Analysis:** Feature importance, confusion matrix, Accuracy, Precision, Recall, and F1 Score were used to analyse model performance.

4.4 K-Nearest Neighbors (KNN)

- **Algorithm:** K-Nearest Neighbors —
sklearn.neighbors.KNeighborsClassifier
- **How it works:** KNN classifies a new student by finding the most similar students in the training dataset. The classes of the nearest neighbours are considered to determine whether the new student is likely to **Pass or Fail**.
- **Purpose:** KNN was included as a **distance-based classification algorithm** to determine whether students with similar academic and behavioural characteristics have similar outcomes.
- **Scaling:** Feature scaling using StandardScaler is important for KNN because the algorithm calculates distances between observations. Without scaling, features with larger numerical ranges could dominate the distance calculation.
- **Hyperparameter Tuning:** The following parameters were evaluated:
 - Number of neighbours (n_neighbors)
 - Distance weighting (weights)
 - Distance metric (metric)
- **Best Configuration:** Cross-validation selected n_neighbors = 5, weights = 'distance', and the Manhattan distance metric as the best configuration. The best cross-validation accuracy was 52.95%.
- **Model-Specific Analysis:** Accuracy, Precision, Recall, F1 Score, and confusion matrix were used to evaluate the classification performance.

4.5 Support Vector Machine (SVM)

- **Algorithm:** Support Vector Machine — `sklearn.svm.SVC`
- **How it works:** SVM attempts to find a decision boundary that separates the two student outcome classes. The algorithm identifies a boundary that maximizes the margin between the classes. A kernel can be used to model non-linear relationships between the input features.
- **Purpose:** SVM was included as a **kernel-based classification algorithm** to compare its performance with linear, tree-based, ensemble, and distance-based models.
- **Scaling:** Features were standardized using `StandardScaler` because SVM is sensitive to feature magnitude and the geometry of the feature space.
- **Hyperparameters:** Important SVM parameters include:
 - kernel
 - C
 - gamma
- **Model-Specific Analysis:** The SVM model was evaluated using Accuracy, Precision, Recall, F1 Score, confusion matrix, and classification report.

5. Evaluation Metrics

All five machine learning models were evaluated on the **held-out test set** using the following classification metrics:

Metric	Formula	Interpretation
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Measures the overall percentage of correctly classified students. Higher accuracy indicates better overall classification performance.
Precision	$TP / (TP + FP)$	Measures how many students predicted as Passed actually passed. Higher precision means fewer false-positive predictions.
Recall	$TP / (TP + FN)$	Measures how many students who actually Passed were correctly identified. Higher recall means fewer students who passed were missed.
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of precision and recall. It provides a balanced measure when both false

		positives and false negatives are important.
Confusion Matrix	[[TN, FP], [FN, TP]]	Shows the number of correct and incorrect predictions for both Passed and Not Passed classes. It helps identify the type of classification errors made by the model.

Table 6: Evaluation Metrics (Formula & Interpretation)

6. Technology Stack

Component	Technology
Programming Language	Python 3.x
Data Manipulation	pandas, NumPy
Machine Learning	scikit-learn
Machine Learning Algorithms	Logistic Regression, Decision Tree, Random Forest, KNN, SVM
Data Preprocessing	pandas, scikit-learn
Feature Scaling	StandardScaler
Visualization	Matplotlib, Seaborn
Model Evaluation	scikit-learn (Accuracy, Precision, Recall, F1-Score, Confusion Matrix)
Model Persistence	joblib
Notebook Environment	Jupyter Notebook (.ipynb)
Dataset Format	CSV

Table 7: Technology Stack (Component & Library)

7. CODE

```
# 1. IMPORT LIBRARIES
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix, classification_report

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
# 2. LOAD DATASET
DATASET_PATH = "student_performance_prediction_50000_records_10_columns.csv"
df = pd.read_csv(DATASET_PATH)

print("Dataset loaded successfully!")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

```
display(df.head())
display(df.tail())

print("Columns:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
display(df.describe(include="all"))

print("\nStudent ID check:")
print("Total Student IDs:", df["Student ID"].nunique())
print("Total Records:", len(df))
```

```

print("Missing values before cleaning:")
display(df.isnull().sum())

print("Duplicate records:", df.duplicated().sum())

df = df.drop_duplicates().copy()
df = df.dropna(subset=["Passed"]).copy()

X = df.drop(["Student ID", "Passed"], axis=1).copy()
y = df["Passed"].copy()

numerical_columns = [
    "Study Hours per Week", "Attendance Rate", "Previous Grades",
    "Assignment Completion Rate", "Mock Test Score", "Daily Self-Study Hours"
]
categorical_columns = [
    "Participation in Extracurricular Activities", "Parent Education Level"
]

for col in numerical_columns:
    X[col] = X[col].fillna(X[col].median())

for col in categorical_columns:
    X[col] = X[col].fillna(X[col].mode()[0])

print("\nMissing values after cleaning:")
display(X.isnull().sum())
print("Usable records:", len(X))

```

```

print("Pass/Fail distribution:")
display(df["Passed"].value_counts())

plt.figure(figsize=(7,5))
df["Passed"].value_counts().plot(kind="bar")
plt.title("Pass/Fail Distribution")
plt.xlabel("Result")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

plt.figure(figsize=(7,7))
df["Passed"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Pass/Fail Percentage")
plt.ylabel("")
plt.show()

```

```

for col in numerical_columns:
    plt.figure(figsize=(7,5))
    plt.hist(X[col], bins=20)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

```

```

# Boxplots: numerical features vs Pass/Fail
eda_temp = X.copy()
eda_temp["Passed"] = y.values

for col in numerical_columns:
    eda_temp.boxplot(column=col, by="Passed", figsize=(7,5))
    plt.title(f"{col} vs Pass/Fail")
    plt.suptitle("")
    plt.xlabel("Passed")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()

```

```

feature_encoders = {}

for col in categorical_columns:
    encoder = LabelEncoder()
    X[col] = encoder.fit_transform(X[col].astype(str))
    feature_encoders[col] = encoder

target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y.astype(str))

print("Target classes:", target_encoder.classes_)
print("Unique encoded target values:", np.unique(y))
print("\nTarget distribution:")
display(pd.Series(y).value_counts().sort_index())

print("\nFeatures used:")
print(X.columns.tolist())

```

```

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

```

```

print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)

# Scaling for Logistic Regression, KNN and SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaled training data:", X_train_scaled.shape)
print("Scaled testing data :", X_test_scaled.shape)
print("NaN in scaled training:", np.isnan(X_train_scaled).sum())
print("NaN in scaled testing :", np.isnan(X_test_scaled).sum())

```

```

# 1. Logistic Regression
lr_model = LogisticRegression(random_state=42, max_iter=5000)
lr_model.fit(X_train_scaled, y_train)
lr_pred = lr_model.predict(X_test_scaled)

# 2. Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

# 3. Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

# 4. KNN
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train)
knn_pred = knn_model.predict(X_test_scaled)

# 5. SVM
svm_model = SVC(kernel="rbf", random_state=42)
svm_model.fit(X_train_scaled, y_train)
svm_pred = svm_model.predict(X_test_scaled)

print("All five models trained successfully!")

```

```

predictions = {
    "Logistic Regression": lr_pred,
    "Decision Tree": dt_pred,
    "Random Forest": rf_pred,
    "KNN": knn_pred,
    "SVM": svm_pred
}

```

```

results_list = []

for name, pred in predictions.items():
    acc = accuracy_score(y_test, pred)
    prec = precision_score(y_test, pred, zero_division=0)
    rec = recall_score(y_test, pred, zero_division=0)
    f1 = f1_score(y_test, pred, zero_division=0)

    results_list.append([name, acc, prec, rec, f1])

    print("=" * 65)
    print(name)
    print("=" * 65)
    print("Accuracy :", round(acc*100, 2), "%")
    print("Precision:", round(prec*100, 2), "%")
    print("Recall   :", round(rec*100, 2), "%")
    print("F1 Score :", round(f1*100, 2), "%")
    print("\nConfusion Matrix:")
    print(confusion_matrix(y_test, pred))
    print("\nClassification Report:")
    print(classification_report(
        y_test, pred,
        target_names=target_encoder.classes_,
        zero_division=0
    ))

```

```

results = pd.DataFrame(
    results_list,
    columns=["Algorithm", "Accuracy", "Precision", "Recall", "F1 Score"]
)

display(results.style.format({
    "Accuracy": "{:.2%}",
    "Precision": "{:.2%}",
    "Recall": "{:.2%}",
    "F1 Score": "{:.2%}"
})))

```

```

plt.figure(figsize=(10,6))
plt.bar(results["Algorithm"], results["Accuracy"] * 100)
plt.title("Accuracy Comparison of Classification Algorithms")
plt.xlabel("Algorithm")
plt.ylabel("Accuracy (%)")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

```

```

fig, axes = plt.subplots(2, 3, figsize=(13, 8))
axes = axes.flatten()

for i, (name, pred) in enumerate(predictions.items()):
    cm = confusion_matrix(y_test, pred)
    axes[i].imshow(cm)
    axes[i].set_title(name)
    axes[i].set_xlabel("Predicted")
    axes[i].set_ylabel("Actual")
    axes[i].set_xticks([0, 1])
    axes[i].set_yticks([0, 1])
    for r in range(2):
        for c in range(2):
            axes[i].text(c, r, cm[r, c], ha="center", va="center")

axes[-1].axis("off")
plt.tight_layout()
plt.show()

```

```

best_index = results["F1 Score"].idxmax()
best_algorithm = results.loc[best_index, "Algorithm"]

print("Best Model:", best_algorithm)
print("Best Accuracy:", round(results.loc[best_index, "Accuracy"]*100, 2),
      "%")
print("Best F1 Score:", round(results.loc[best_index, "F1 Score"]*100, 2),
      "%")

# Retrain the selected algorithm on all usable records.
if best_algorithm == "Logistic Regression":
    final_scaler = StandardScaler()
    final_model = LogisticRegression(random_state=42, max_iter=5000)
    final_model.fit(final_scaler.fit_transform(X), y)

elif best_algorithm == "Decision Tree":
    final_scaler = None
    final_model = DecisionTreeClassifier(random_state=42)
    final_model.fit(X, y)

elif best_algorithm == "Random Forest":
    final_scaler = None
    final_model = RandomForestClassifier(n_estimators=100, random_state=42)
    final_model.fit(X, y)

elif best_algorithm == "KNN":
    final_scaler = StandardScaler()

```

```

final_model = KNeighborsClassifier(n_neighbors=5)
final_model.fit(final_scaler.fit_transform(X), y)

else:
    final_scaler = StandardScaler()
    final_model = SVC(kernel="rbf", random_state=42)
    final_model.fit(final_scaler.fit_transform(X), y)

print("Final model training completed!")

```

```

# Evaluate the selected model on the held-out test set.
if best_algorithm == "Logistic Regression":
    final_test_pred = final_model.predict(final_scaler.transform(X_test))
elif best_algorithm in ["Decision Tree", "Random Forest"]:
    final_test_pred = final_model.predict(X_test)
else:
    final_test_pred = final_model.predict(final_scaler.transform(X_test))

final_accuracy = accuracy_score(y_test, final_test_pred)
final_precision = precision_score(y_test, final_test_pred, zero_division=0)
final_recall = recall_score(y_test, final_test_pred, zero_division=0)
final_f1 = f1_score(y_test, final_test_pred, zero_division=0)

print("Final Model:", best_algorithm)
print("Accuracy :", round(final_accuracy*100, 2), "%")
print("Precision:", round(final_precision*100, 2), "%")
print("Recall   :", round(final_recall*100, 2), "%")
print("F1 Score :", round(final_f1*100, 2), "%")

final_cm = confusion_matrix(y_test, final_test_pred)
print("\nFinal Confusion Matrix:")
print(final_cm)

print("\nFinal Classification Report:")
print(classification_report(
    y_test, final_test_pred,
    target_names=target_encoder.classes_,
    zero_division=0
))

```

```

# Final confusion matrix
plt.figure(figsize=(6,5))
plt.imshow(final_cm)
plt.title(f"Final Confusion Matrix - {best_algorithm}")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks([0,1], target_encoder.classes_)

```

```

plt.yticks([0,1], target_encoder.classes_)

for r in range(2):
    for c in range(2):
        plt.text(c, r, final_cm[r,c], ha="center", va="center")

plt.tight_layout()
plt.show()

# Final bar graph
final_metrics = pd.Series({
    "Accuracy": final_accuracy,
    "Precision": final_precision,
    "Recall": final_recall,
    "F1 Score": final_f1
})

plt.figure(figsize=(8,5))
plt.bar(final_metrics.index, final_metrics.values*100)
plt.title("Final Model Performance")
plt.ylabel("Score (%)")
plt.ylim(0,100)
plt.tight_layout()
plt.show()

# Final line chart
plt.figure(figsize=(8,5))
plt.plot(final_metrics.index, final_metrics.values*100, marker="o")
plt.title("Final Model Metric Comparison")
plt.xlabel("Metric")
plt.ylabel("Score (%)")
plt.ylim(0,100)
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

def create_student_features(
    study_hours, attendance, previous_grades, extracurricular,
    parent_education, assignment_completion, mock_test_score,
    self_study_hours
):
    new_student = pd.DataFrame({
        "Study Hours per Week": [study_hours],
        "Attendance Rate": [attendance],
        "Previous Grades": [previous_grades],
        "Participation in Extracurricular Activities": [extracurricular],
        "Parent Education Level": [parent_education],
        "Assignment Completion Rate": [assignment_completion],
    })

```

```

        "Mock Test Score": [mock_test_score],
        "Daily Self-Study Hours": [self_study_hours]
    })

    for col in categorical_columns:
        encoder = feature_encoders[col]
        value = str(new_student.loc[0, col])
        if value not in encoder.classes_:
            raise ValueError(
                f"Unknown {col}: {value}. Allowed values:
{list(encoder.classes_)}"
            )
        new_student[col] = encoder.transform([value])

    return new_student[X.columns]

```

```

new_student_features = create_student_features(
    study_hours=20,
    attendance=88,
    previous_grades=80,
    extracurricular="Yes",
    parent_education="Bachelor",
    assignment_completion=90,
    mock_test_score=85,
    self_study_hours=3
)

print("New student's extracted features:")
display(new_student_features)

```

```

if final_scaler is not None:
    prediction_input = final_scaler.transform(new_student_features)
else:
    prediction_input = new_student_features

prediction = final_model.predict(prediction_input)[0]
predicted_result = target_encoder.inverse_transform([prediction])[0]

print("Predicted Result:", predicted_result)

if predicted_result == "Yes":
    print("Prediction: PASS")
else:
    print("Prediction: FAIL")

```

```

test_students = [
    {
        "study_hours": 25, "attendance": 92, "previous_grades": 88,
        "extracurricular": "Yes", "parent_education": "Bachelor",
        "assignment_completion": 95, "mock_test_score": 90,
        "self_study_hours": 4
    },
    {
        "study_hours": 8, "attendance": 55, "previous_grades": 45,
        "extracurricular": "No", "parent_education": "High School",
        "assignment_completion": 50, "mock_test_score": 42,
        "self_study_hours": 1
    }
]

for i, student in enumerate(test_students, start=1):
    features = create_student_features(**student)
    if final_scaler is not None:
        features_for_model = final_scaler.transform(features)
    else:
        features_for_model = features
    pred = final_model.predict(features_for_model)[0]
    result = target_encoder.inverse_transform([pred])[0]
    print(f"Student {i}: {result}")

```

8. Visualization

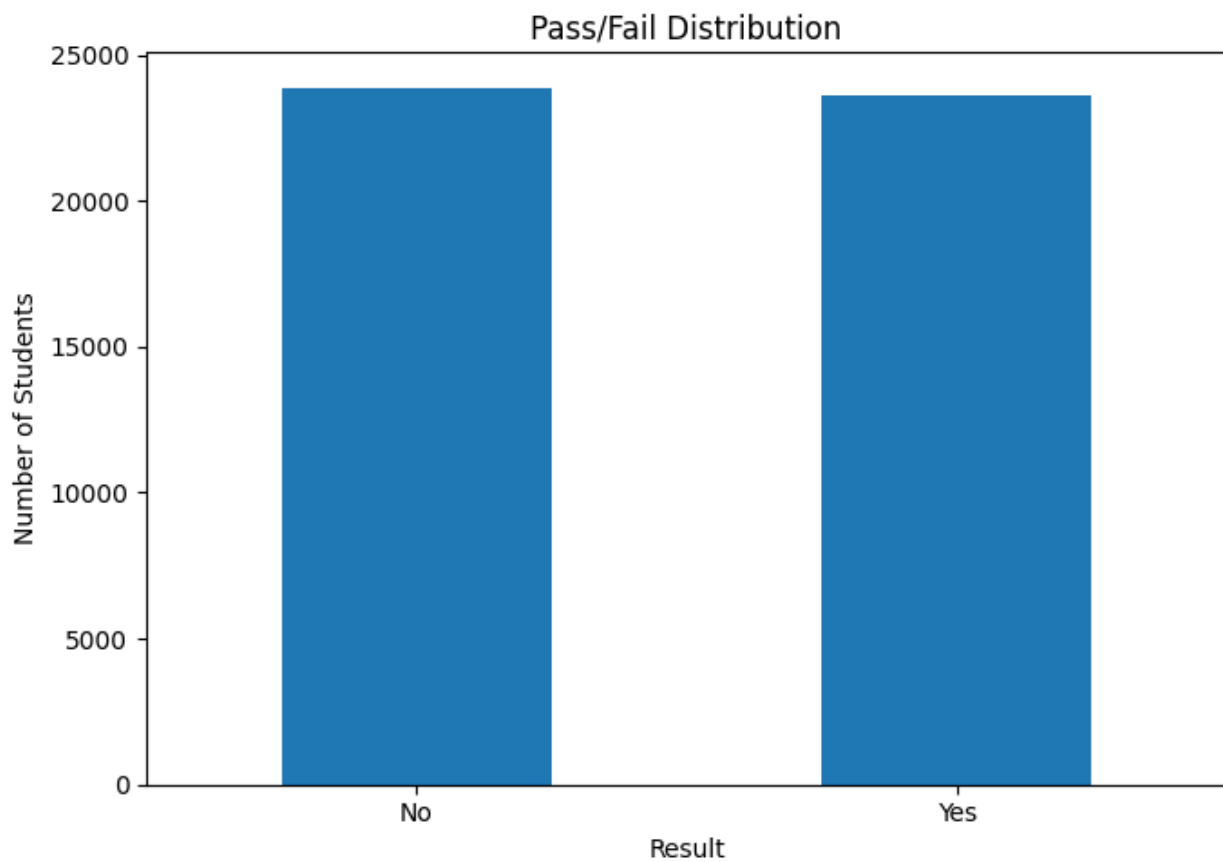


Figure 1: Pass/Fail Distribution

Pass/Fail Percentage

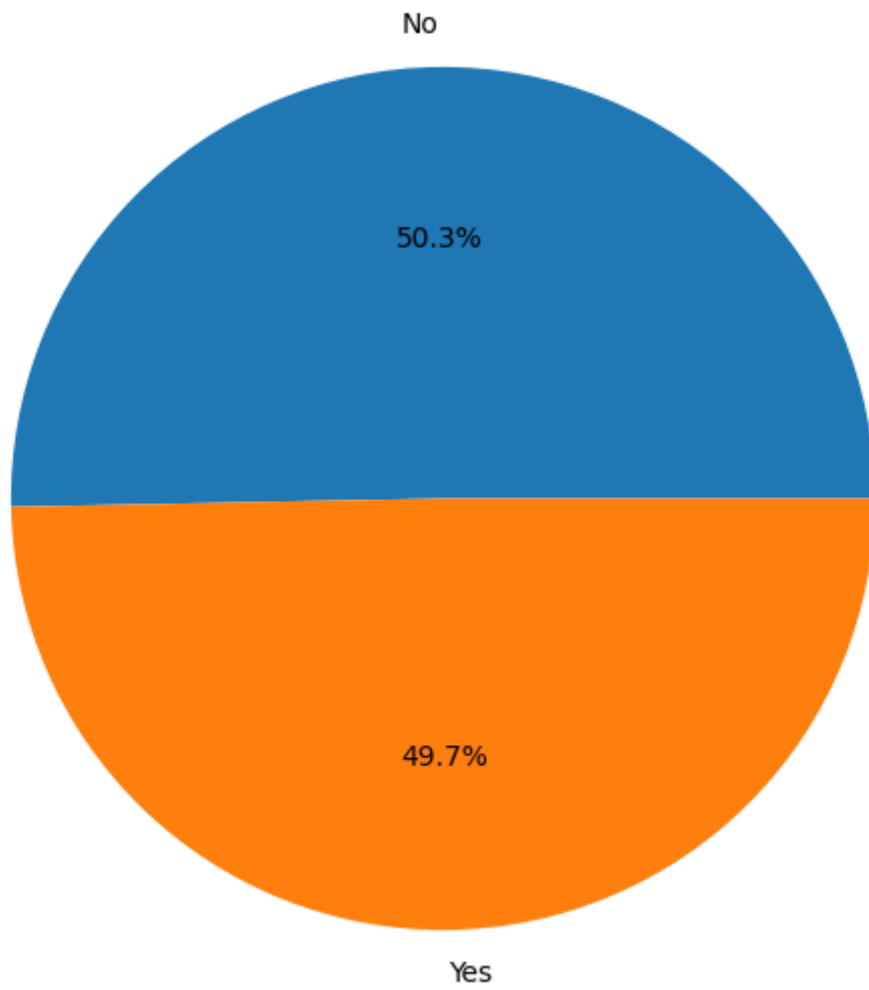
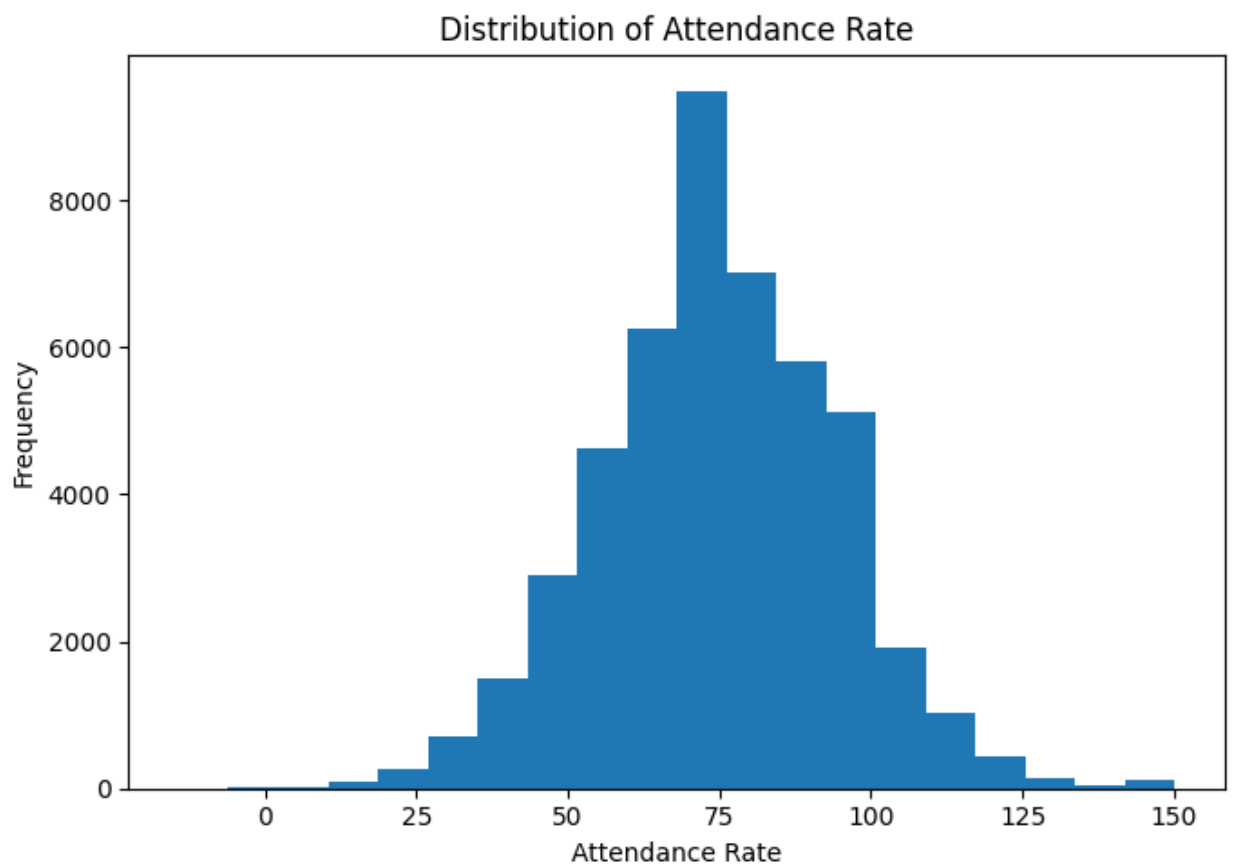
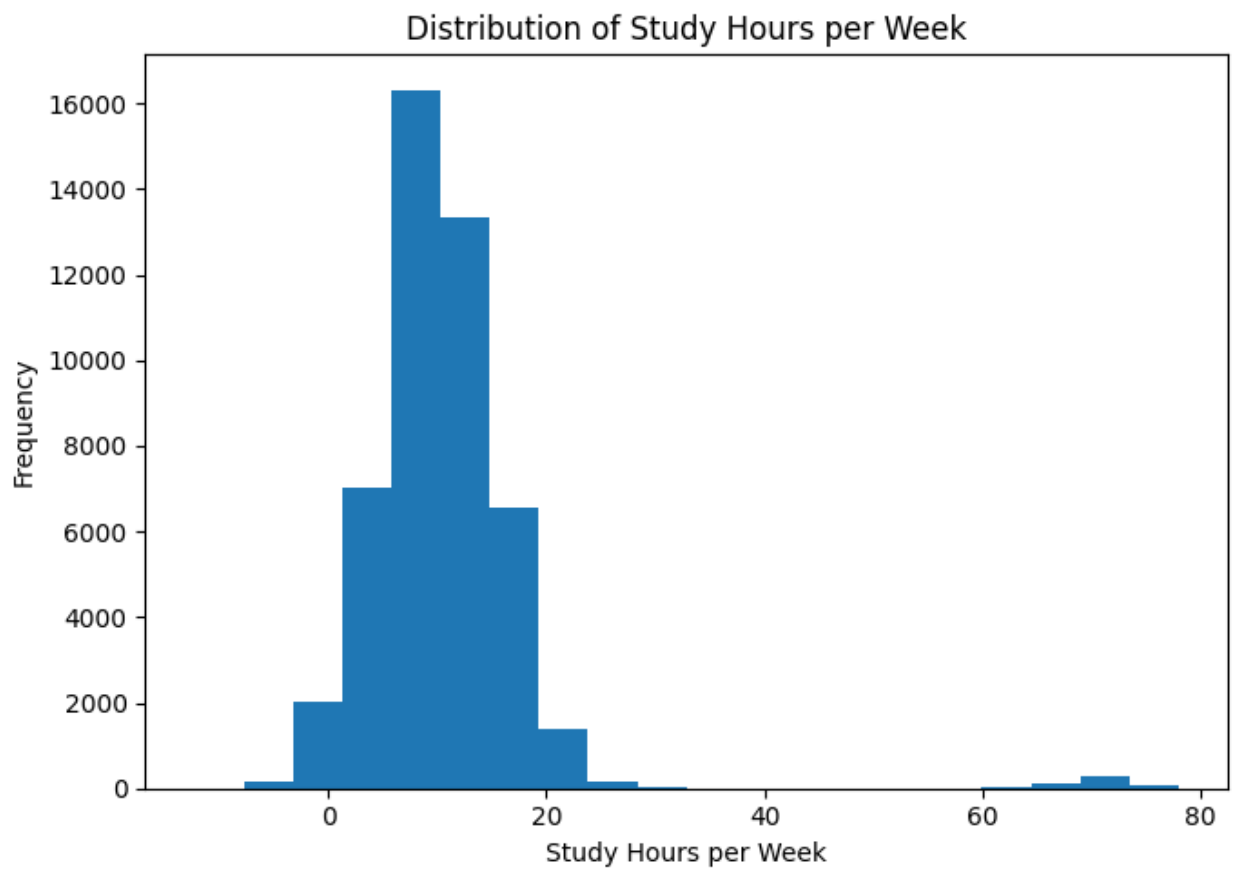
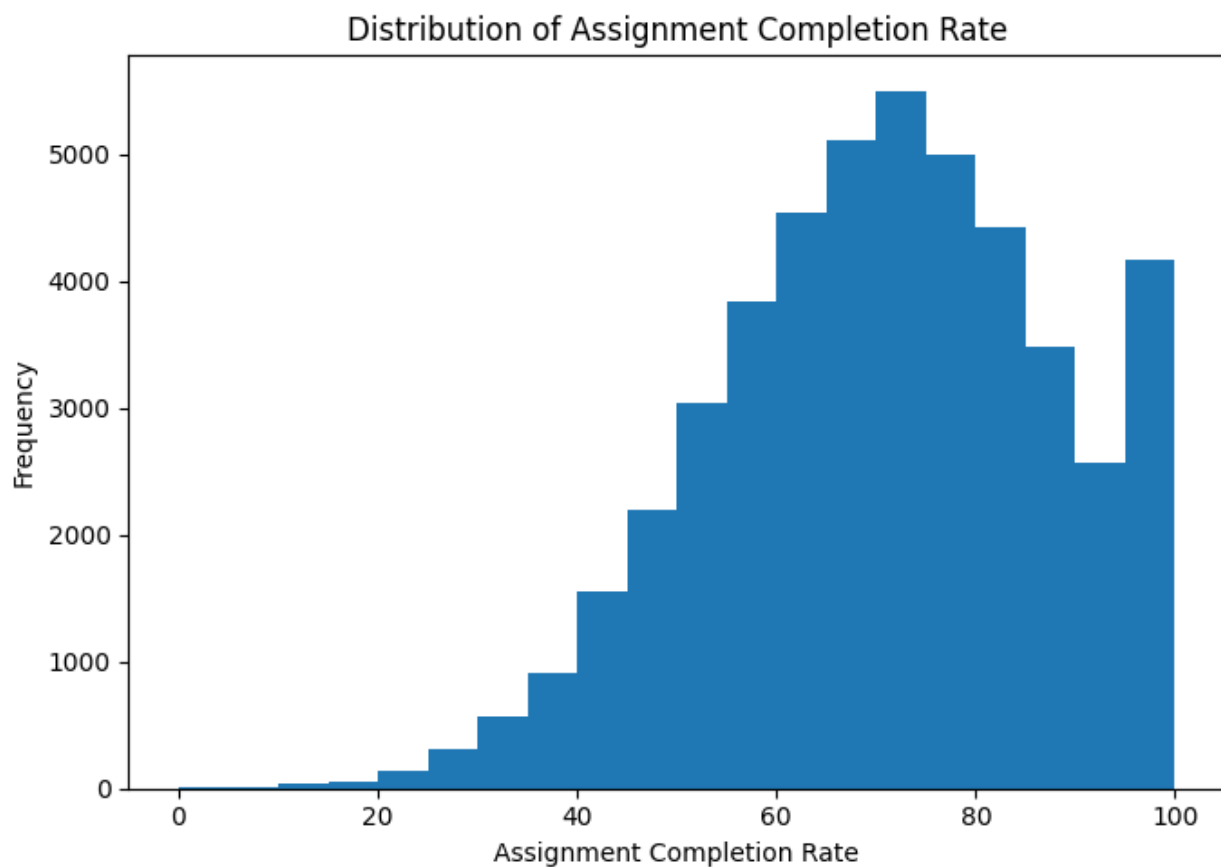
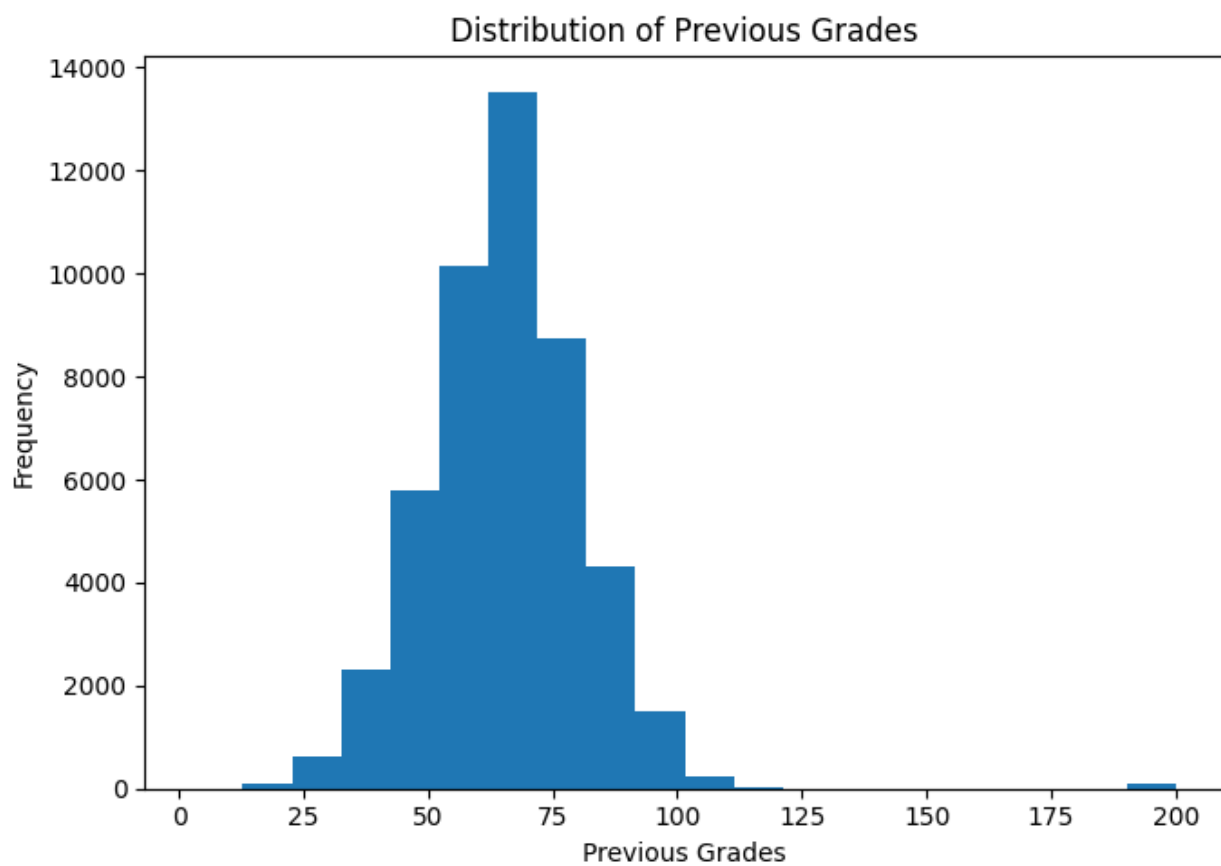


Figure 2: Pass/Fail Percentage





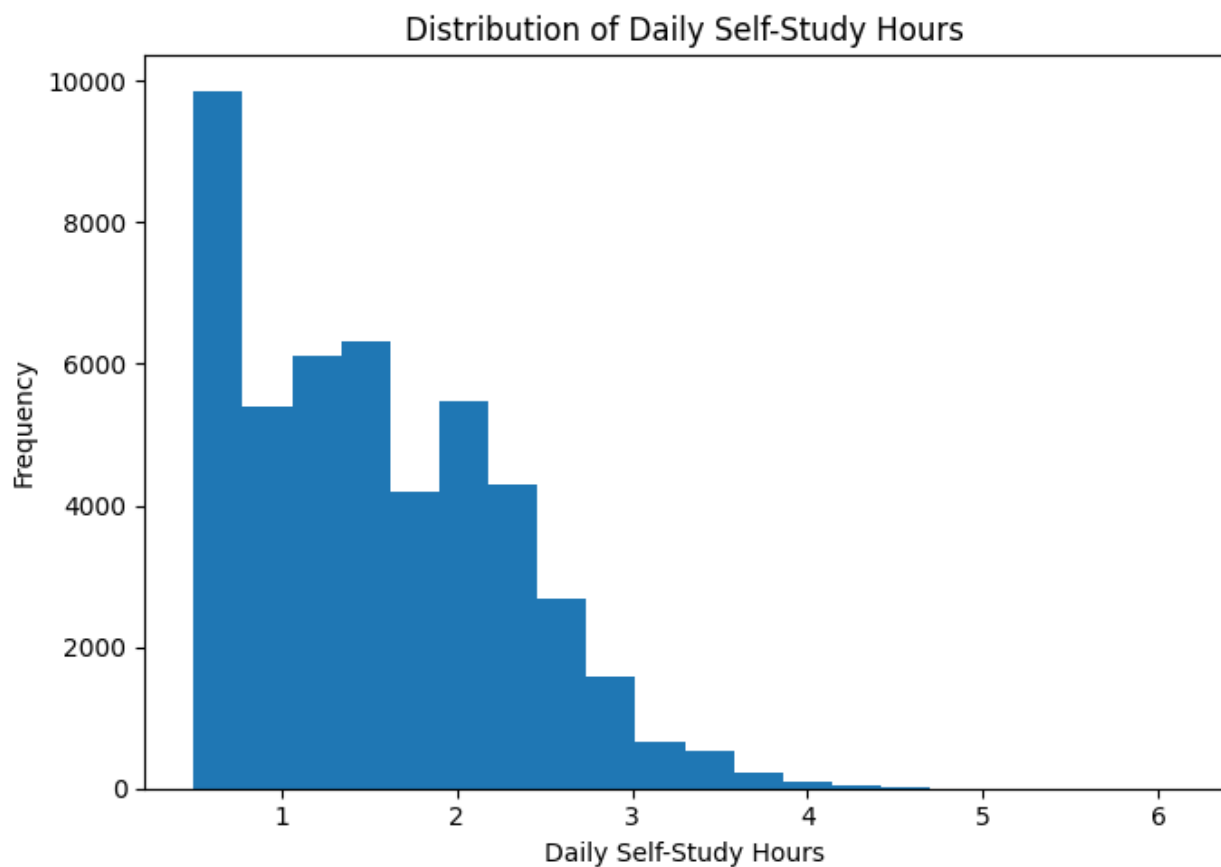
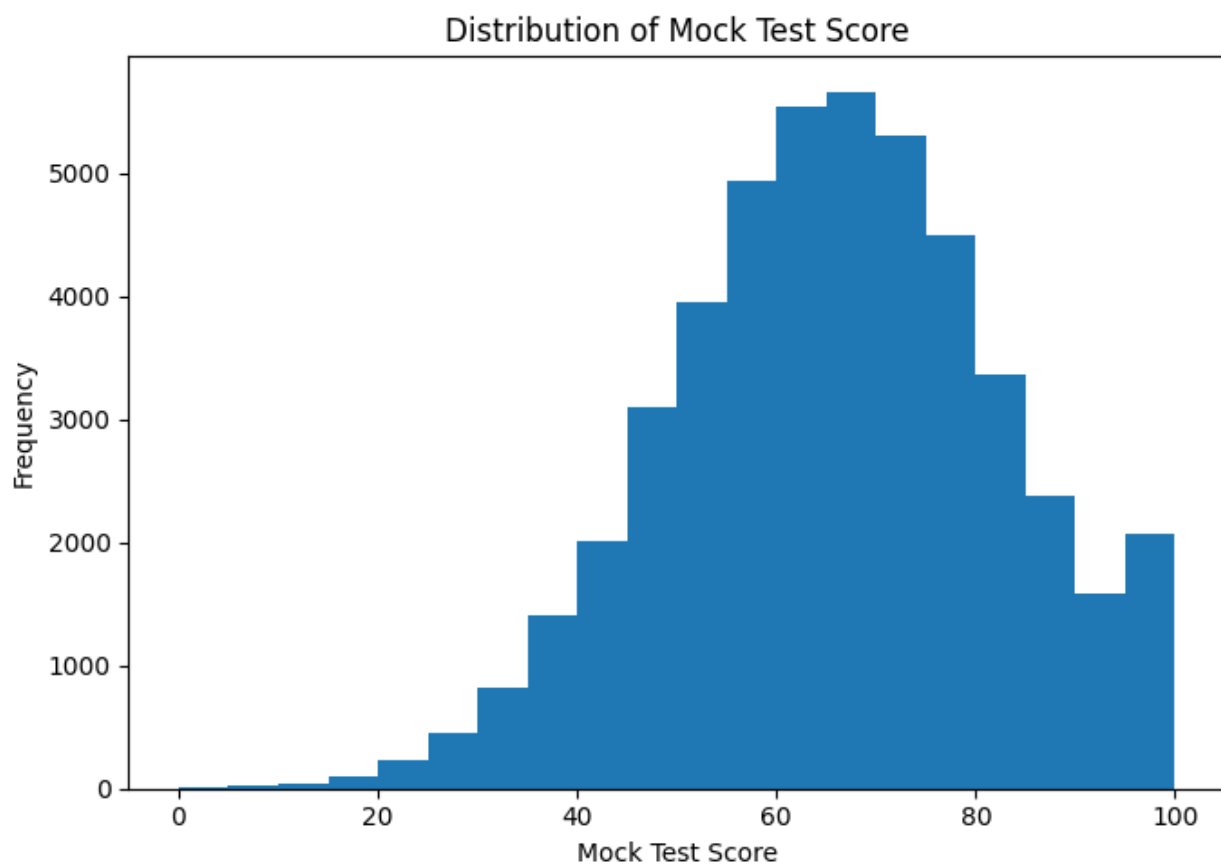
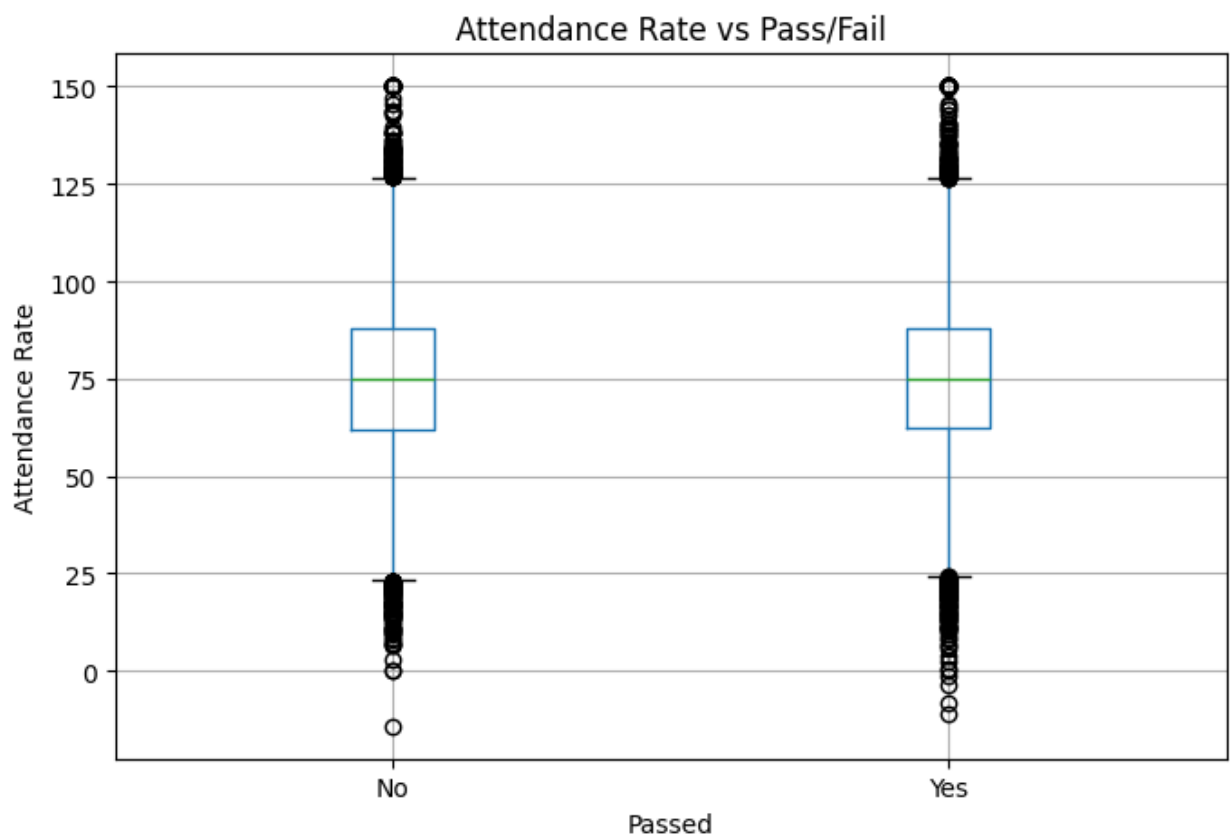
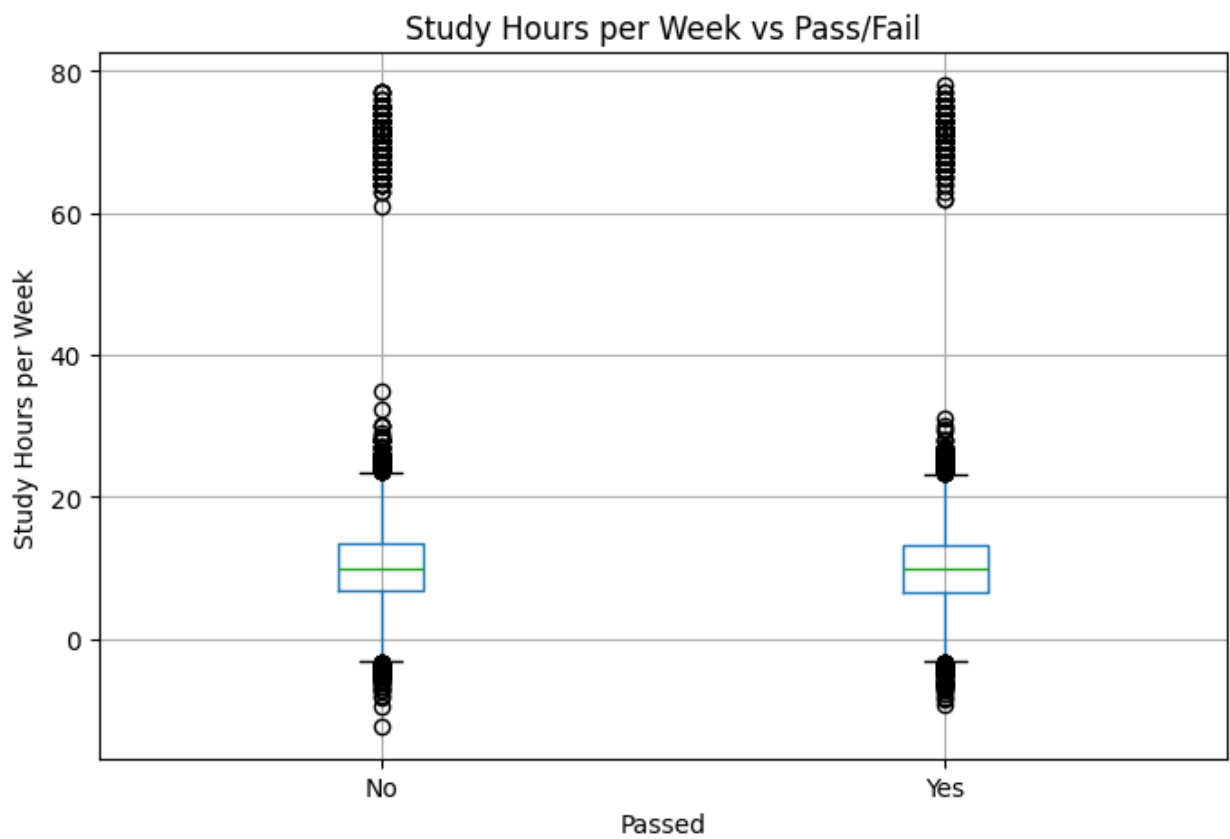
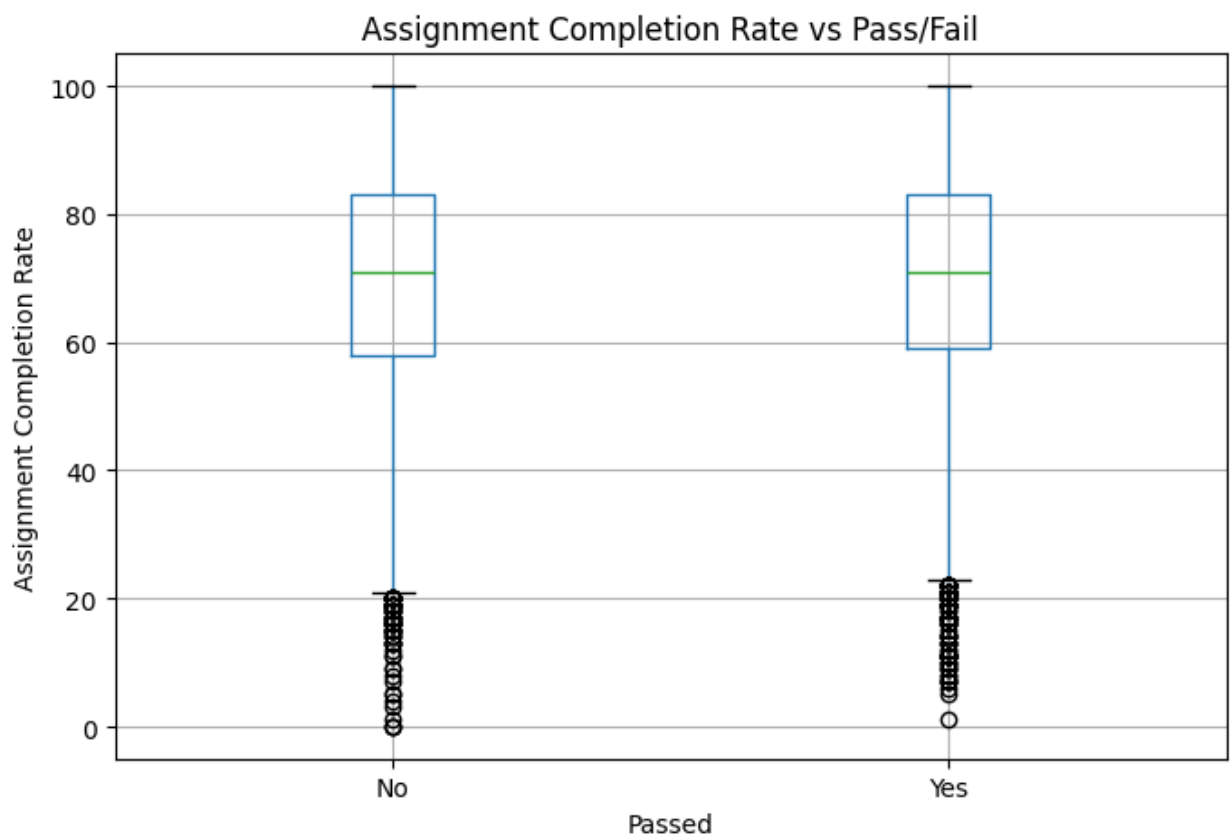
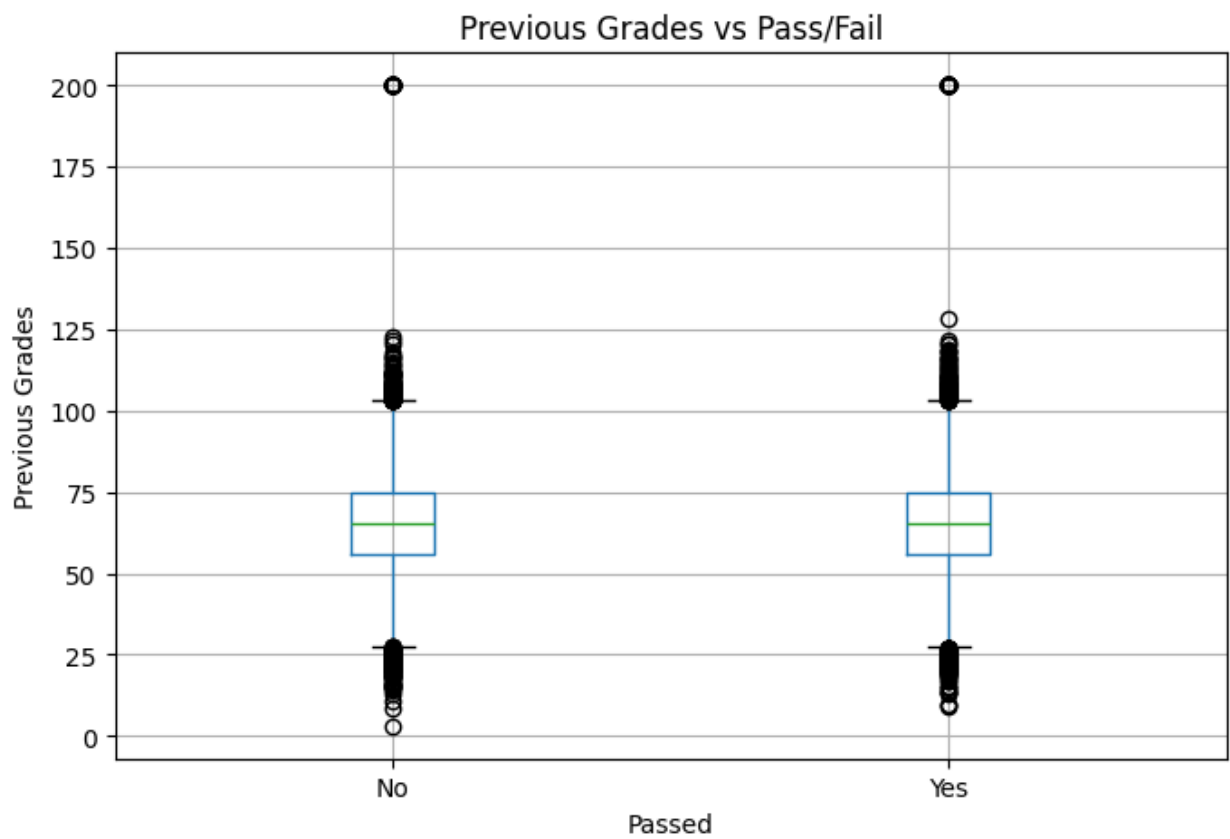


Figure 3: Distribution of Numerical Features





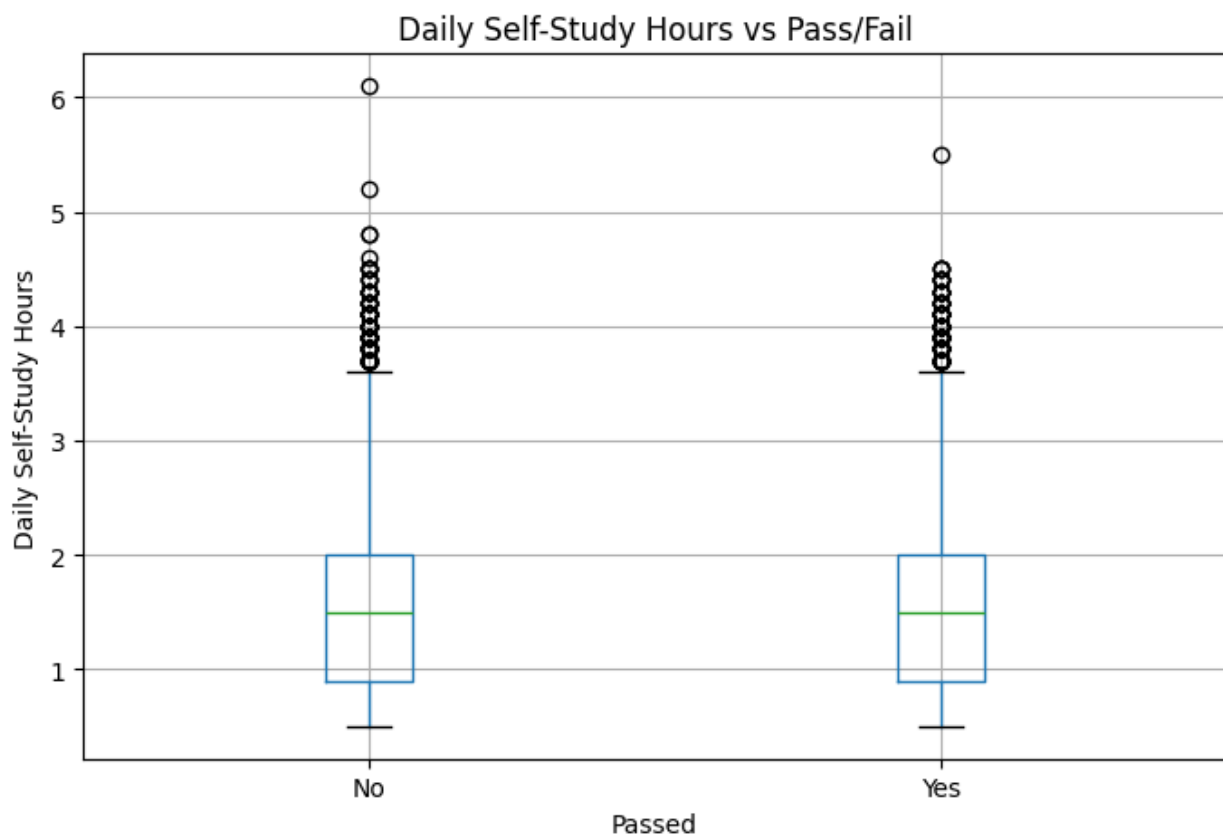
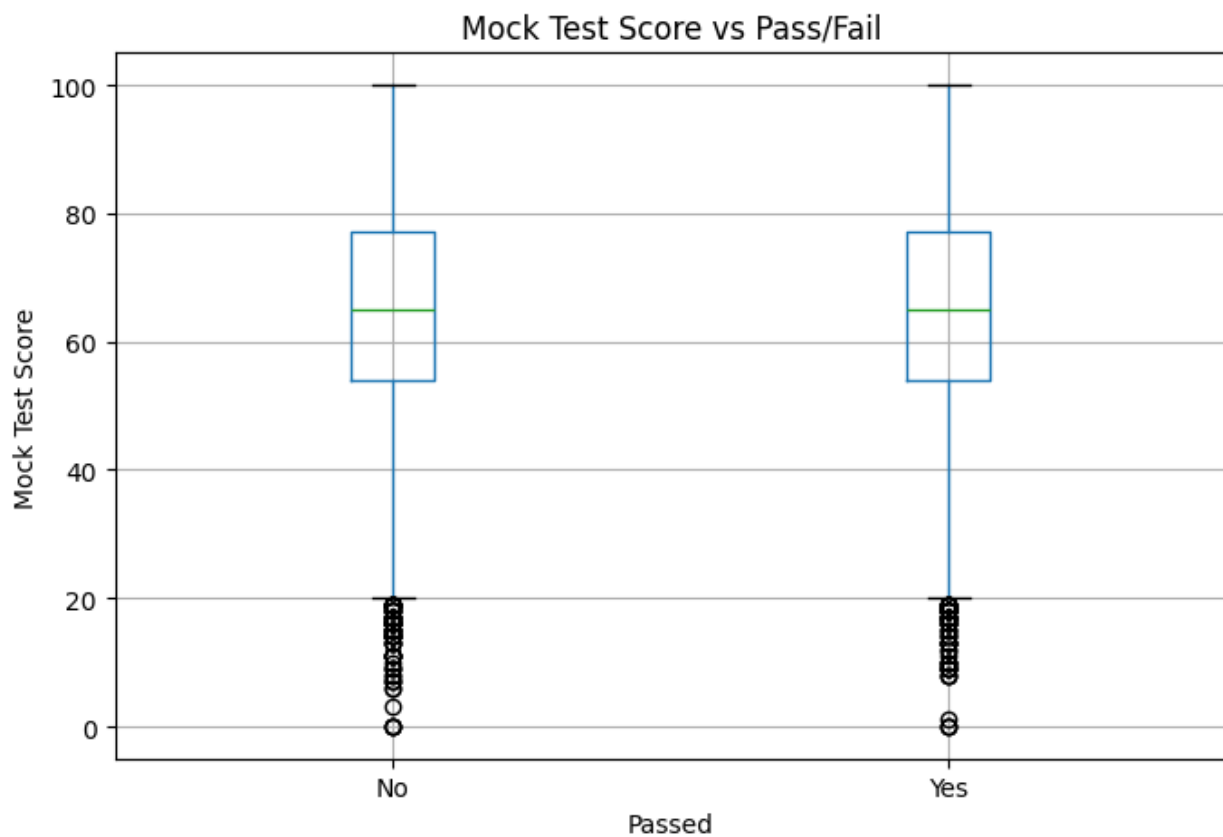


Figure 4: Numerical Features vs Pass/Fail — Boxplots

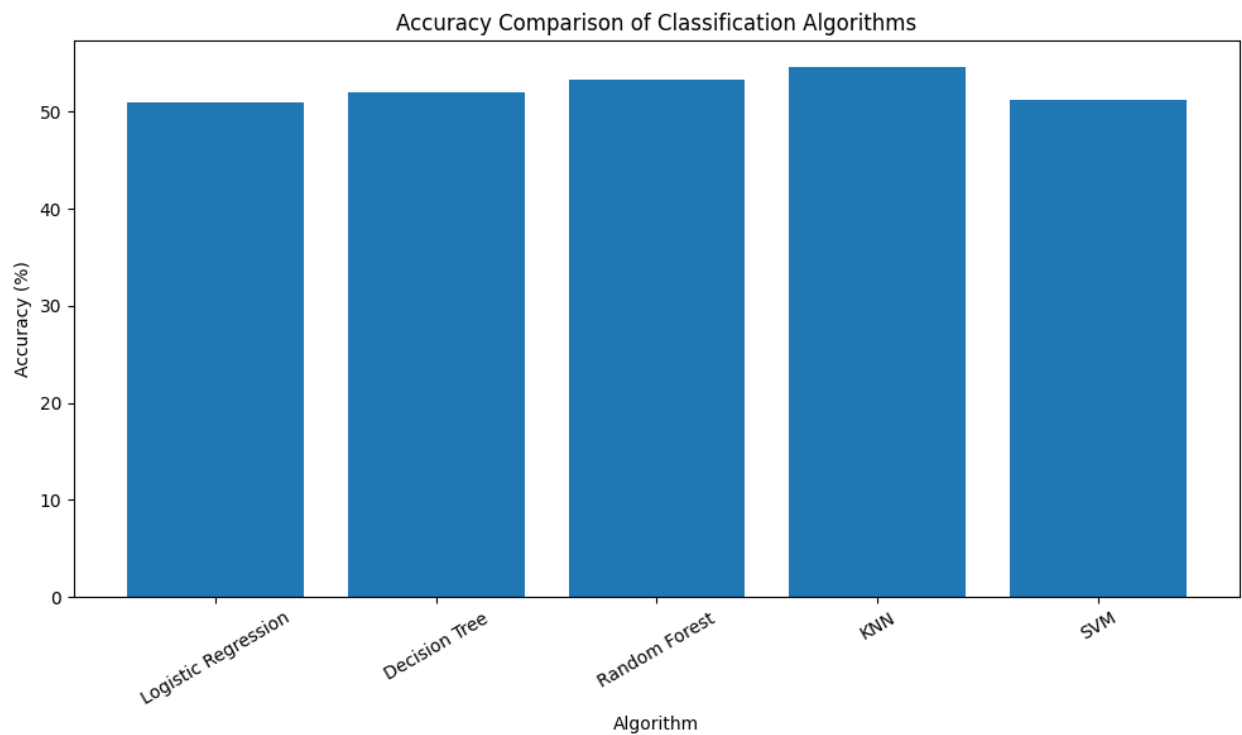


Figure 5: Accuracy Comparison of Classification Algorithms

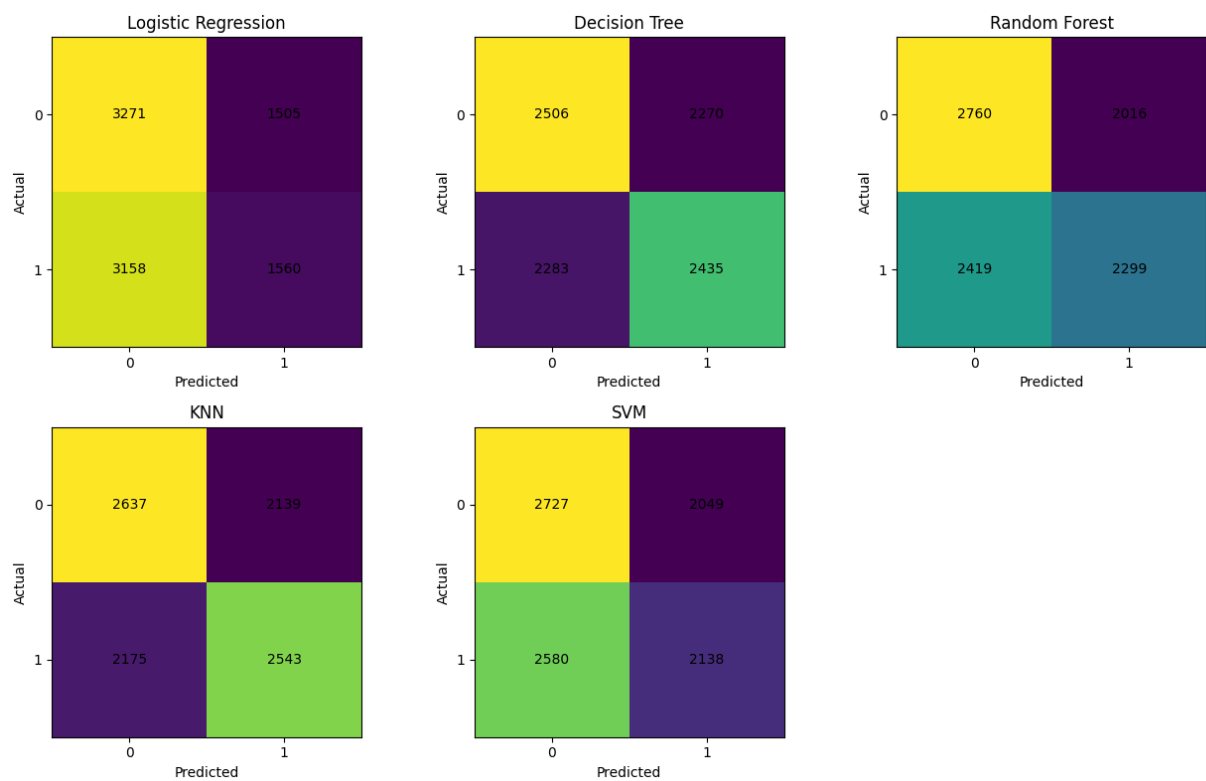


Figure 6: Confusion Matrices of All Five Classification Algorithms

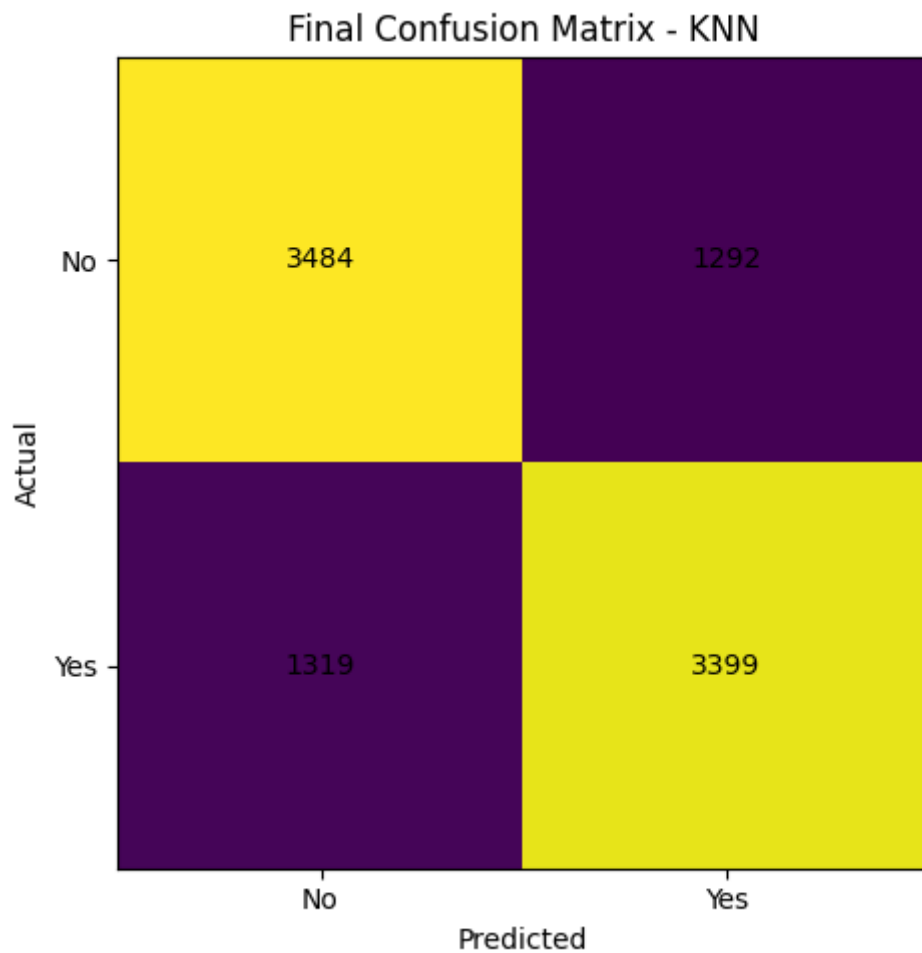


Figure 7: Final Confusion Matrix of Best Performing Model

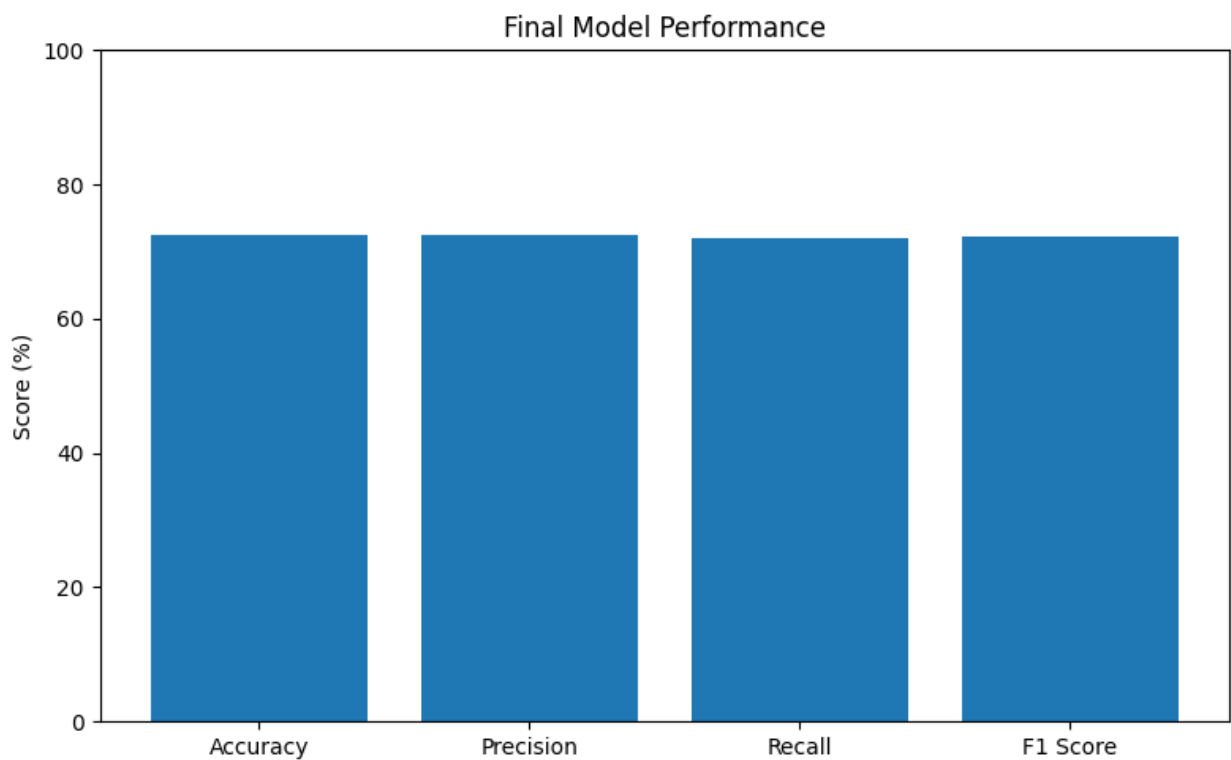


Figure 8: Final Model Performance

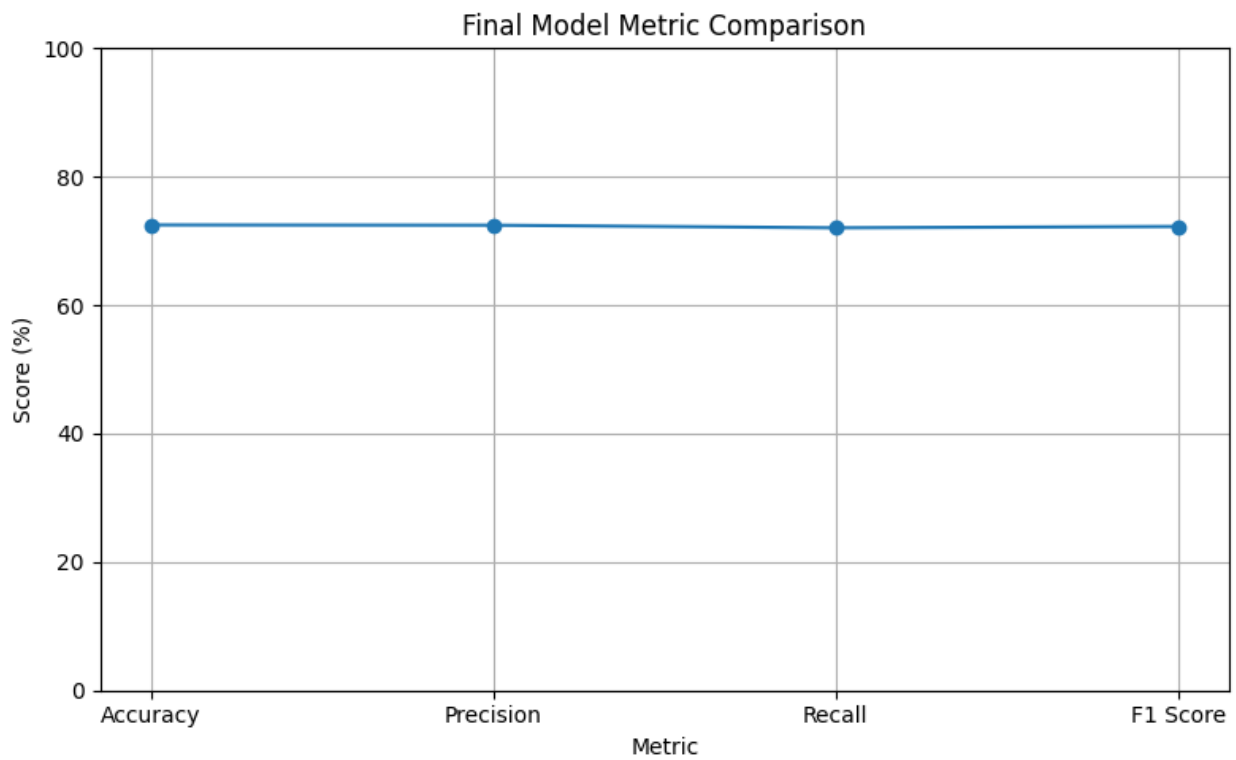


Figure 9: Final Model Metric Comparison

9. Conclusion

This project demonstrates a complete Machine Learning pipeline for predicting **student academic performance** using a binary classification approach. The system involves data preprocessing, handling missing and invalid values, categorical encoding, feature engineering, train-test splitting, model training, and performance evaluation.

Five different classification algorithms — **Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)** — were implemented and evaluated on the same held-out test dataset. The models were compared using **Accuracy, Precision, Recall, F1-Score, and Confusion Matrix** to understand their classification performance.

Key Takeaways:

- **K-Nearest Neighbors (KNN)** achieved the best overall performance among the five models, with an accuracy of approximately **54.56%** on the held-out test set.
- **Random Forest** provided the second-best performance, with an accuracy of approximately **53.29%**, showing that ensemble-based learning can capture some non-linear relationships between student-related features.
- **Decision Tree** achieved approximately **52.04% accuracy**, while **SVM** achieved approximately **51.24% accuracy**.
- **Logistic Regression** achieved approximately **50.88% accuracy** and provided a useful baseline for comparison.
- The results show that no model achieved very high predictive performance, indicating that the available student-related features may have limited ability to explain the target variable or that additional relevant features could improve prediction.

- Feature engineering and preprocessing helped prepare the dataset for reliable model training and evaluation.
- The project demonstrates how different Machine Learning algorithms can be applied and compared to identify the most suitable model for **student performance prediction**.

Overall, the project provides a practical framework for predicting whether a student is likely to **Pass or Not Pass** based on academic and behavioral factors. The results also highlight the importance of proper preprocessing, feature engineering, model comparison, and evaluation on an **unseen test dataset** when developing a classification system.